

DevOps Engineer

Interview Preparation Guide

Prepared for:	Sanket Ajay Chopade
Role:	DevOps Engineer (Entry Level)
Experience:	9-Month Internship at Hisan Labs
Focus Areas:	AWS · Kubernetes · Terraform · Jenkins · Docker · GitOps

This guide covers the most probable technical and behavioural questions based on your resume profile. Answers are tailored to your real-world internship experience and projects.

■ 1. Resume & Introduction Questions

Q: Tell me about yourself.

- I am a B.Tech graduate from RTM Nagpur University with a strong focus on DevOps engineering.
- I completed a 9-month internship at Hisan Labs in Pune, where I worked as a DevOps Engineer Intern on a microservices-based e-commerce application hosted on AWS.
- My core strengths include AWS (EKS, IAM, VPC, RDS, S3, ALB), Terraform for IaC, Kubernetes for container orchestration, and CI/CD pipelines using Jenkins and GitHub Actions.
- I have hands-on experience with GitOps using Argo CD, security integration via SonarQube and Trivy, and monitoring with Prometheus, Grafana, Datadog, and CloudWatch.
- I am passionate about automating infrastructure, improving deployment reliability, and building scalable cloud-native systems.

■ Resume context: Hisan Labs | Sep 2025 – May 2026 | Microservices e-commerce on AWS

Q: Walk me through your DevOps journey.

- My journey started during my B.Tech where I developed a strong interest in cloud and automation.
 - I began learning Linux, Git, and Docker fundamentals before advancing to Kubernetes and cloud platforms.
 - At Hisan Labs, I transitioned from theory to practice — designing Jenkins pipelines, writing Terraform modules, and managing EKS clusters in a real production-like environment.
 - My Two key projects — a Three-Tier DevSecOps deployment and a GitOps delivery pipeline — represent the culmination of this learning journey.
-

Q: What was your role and day-to-day responsibility at Hisan Labs?

- I was part of a 4-member DevOps team responsible for deploying and operating a microservices-based e-commerce application on AWS.
- Day-to-day tasks included maintaining Jenkins CI/CD pipelines, reviewing Terraform plan outputs, monitoring cluster health via Grafana dashboards, and resolving deployment issues.
- I configured GitHub webhooks for automated builds and integrated SonarQube for code quality gates in the pipeline.
- I also managed AWS IAM roles and policies, maintained Terraform remote state, and wrote Bash/Python scripts to automate recurring operational tasks.

■ Resume context: Designed and maintained 3 Jenkins pipelines; managed Terraform remote state; administered Jenkins plugins and credentials.

Q: Explain the architecture of the microservices application you worked on.

- The application was a full-stack e-commerce platform deployed on Amazon EKS with a microservices architecture.
- The frontend (React) and multiple backend services were containerised using Docker and deployed as Kubernetes Deployments.
- AWS ALB (Application Load Balancer) with the AWS Load Balancer Controller handled external traffic via Kubernetes Ingress.

- Data was persisted using Amazon RDS (relational) and state-sensitive services used Kubernetes StatefulSets.
- Infrastructure was provisioned using Terraform modules — covering VPC, subnets, security groups, EKS, ECR, and RDS.
- The CI/CD flow: code push → GitHub webhook → Jenkins (build, SonarQube scan, Trivy scan, Docker build/push to ECR) → Argo CD syncs to EKS.

■ Resume context: *Three-Tier DevSecOps Project on AWS EKS*

■ 2. Linux Interview Questions

Q: What happens when Linux boots?

- BIOS/UEFI runs POST and locates the bootloader (GRUB).
 - GRUB loads the Linux kernel into memory.
 - The kernel initialises hardware, mounts the root filesystem, and starts the init system (systemd).
 - systemd starts system services and targets in order, culminating in a login prompt.
-

Q: Difference between a process and a thread?

- A process is an independent program in execution with its own memory space.
 - A thread is a lightweight unit of execution within a process, sharing the same memory space.
 - Processes are isolated; a crash in one process does not affect others.
 - Threads within the same process can communicate directly but require synchronisation (mutexes, locks) to avoid race conditions.
-

Q: How do you find high CPU usage processes and kill them?

- Use `top` or `htop` to view real-time CPU usage sorted by highest consumer.
 - Use `ps aux --sort=-%cpu | head -10` for a snapshot of top 10 CPU-consuming processes.
 - Identify the PID (Process ID) from the output.
 - Use `kill` for a graceful termination (SIGTERM) or `kill -9` for a forceful kill (SIGKILL).
 - Use `pkill` to kill by name.
-

Q: Explain: grep, sed, awk — and when to use each.

- `grep`: Searches for patterns in files/output. Use when you need to filter lines. Example: `grep 'ERROR' app.log`
 - `sed` (Stream Editor): Modifies text in a stream/file. Best for find-and-replace. Example: `sed 's/old/new/g' file.txt`
 - `awk`: A full text-processing language. Best for column-based data processing. Example: `awk '{print $1, $3}' log.txt` to print columns 1 and 3.
-

Q: Disk is full (95%). How do you troubleshoot?

- Check overall disk usage: `df -h` to see which filesystem/partition is full.
 - Find large files/directories: `du -sh /* | sort -rh | head -20`
 - Check for large log files: `ls -lh /var/log/`
 - Identify and remove old Docker images/containers: `docker system prune -af`
 - Check for zombie processes with open file handles on deleted files: `ls -l | grep deleted`
 - Rotate or archive old logs; remove stale package caches with `apt clean`.
-

Q: Difference between soft link and hard link?

- Hard link: A direct reference to the inode (actual data on disk). Deleting the original file does not remove the data as long as a hard link exists.

- Soft link (Symbolic link): A pointer to the file path, not the inode. If the original file is deleted, the symlink breaks.
 - Hard links cannot span different filesystems; soft links can.
 - Create with: `ln file hardlink` and `ln -s file softlink`
-

Q: Server is slow — what steps do you take?

- Check CPU: `top` or `htop` — identify any process consuming >80% CPU.
 - Check Memory: `free -h` — look for high usage or heavy swap activity.
 - Check Disk I/O: `iostat -x 1` or `iotop` — identify disk-bound processes.
 - Check Network: `netstat -s` or `ss -s` — look for high connection counts or errors.
 - Check system logs: `journalctl -xe` or `/var/log/syslog` for errors.
 - Check running services: `systemctl list-units --state=failed`
-

■ 3. Docker Interview Questions

Q: VM vs Container — what is the key difference?

- A Virtual Machine (VM) includes a full OS with its own kernel, running on a hypervisor. It is heavy (GBs) and slow to start.
- A Container shares the host OS kernel and packages only the application and its dependencies. It is lightweight (MBs) and starts in seconds.
- VMs provide stronger isolation; containers provide speed, portability, and density.
- In practice: containers are ideal for microservices; VMs are used when full OS isolation or different OS types are needed.

■ *Resume context: Used Docker to containerise frontend and backend services deployed on Amazon EKS.*

Q: Difference between CMD and ENTRYPOINT in a Dockerfile?

- ENTRYPOINT defines the executable that always runs when the container starts. It cannot be easily overridden.
- CMD provides default arguments to the ENTRYPOINT, or the default command if no ENTRYPOINT is set. It can be overridden at ``docker run``.
- Best practice: use ENTRYPOINT for the main process (e.g., the application binary) and CMD for default arguments.
- Example: ``ENTRYPOINT ["python"]`` + ``CMD ["app.py"]`` — running ``docker run image other.py`` overrides CMD but keeps ENTRYPOINT.

Q: What is a multi-stage Docker build and why use it?

- A multi-stage build uses multiple FROM instructions in one Dockerfile, where each stage can copy artifacts from previous stages.
- The final image only contains what is copied into the last stage, discarding build tools, compilers, and intermediate files.
- This dramatically reduces image size and attack surface.
- Example: Stage 1 uses a ``golang:1.21`` image to compile a binary; Stage 2 copies only the binary into a ``distroless`` or ``alpine`` base image.

■ *Resume context: Used multi-stage Dockerfiles in the Go Cloud-Native GitOps project.*

Q: Why use Distroless images?

- Distroless images contain only the application runtime and its direct dependencies — no shell, no package manager, no OS utilities.
- This minimises the attack surface: there is no shell for an attacker to exploit even if they breach the container.
- They are significantly smaller than full OS-based images, improving pull times and registry storage.
- Trivy scans on Distroless images typically show far fewer CVE vulnerabilities.

■ *Resume context: Implemented Distroless images in the Go application project.*

Q: Container exits immediately after starting — how do you debug?

- Check exit code and logs: ``docker logs``
 - Inspect the container state: ``docker inspect`` — look at ``State.ExitCode`` and ``State.Error``.
 - Run interactively to debug: ``docker run -it --entrypoint /bin/sh`` (if shell is available).
 - Common causes: application crash on startup, missing environment variables, wrong CMD/ENTRYPOINT, or missing config files.
 - Check if the base image has the expected binary: ``docker run --rm image which myapp``
-

■ 4. Kubernetes Interview Questions

Q: Explain Kubernetes architecture.

- Control Plane (Master): Manages the cluster state.
 - → API Server: Entry point for all cluster operations; exposes the Kubernetes API.
 - → etcd: Distributed key-value store; source of truth for all cluster data.
 - → Scheduler: Assigns pods to nodes based on resource requirements and constraints.
 - → Controller Manager: Runs controllers (ReplicaSet, Deployment, Node) to reconcile desired vs actual state.
- Worker Nodes: Run the actual application workloads.
 - → Kubelet: Agent on each node; communicates with the API server and manages pod lifecycle.
 - → Kube-proxy: Handles network routing and load balancing for Services.
 - → Container Runtime: (e.g., containerd) Runs containers.

■ Resume context: Managed Amazon EKS clusters; EKS provides a managed control plane.

Q: Difference between Deployment and StatefulSet?

- Deployment: Used for stateless applications. Pods are interchangeable — any pod can be replaced by a new one with a different identity.
- StatefulSet: Used for stateful applications (databases, queues). Each pod has a stable, unique network identity and persistent storage.
- StatefulSet pods are created/deleted in order; Deployment pods are created in parallel.
- Example: Frontend/API services → Deployment. MongoDB, PostgreSQL → StatefulSet.

■ Resume context: Used both Deployments and StatefulSets in the Three-Tier DevSecOps project.

Q: How does Horizontal Pod Autoscaler (HPA) work?

- HPA automatically scales the number of pod replicas based on observed metrics (CPU, memory, or custom metrics).
- It queries the Metrics Server periodically (default: every 15 seconds) and compares current usage against the target threshold.
- If current CPU > target %, HPA increases replicas. If current CPU < target %, it scales down (respecting minReplicas).
- Configuration: `kubectl autoscale deployment myapp --cpu-percent=50 --min=2 --max=10``
- Requires Metrics Server to be installed in the cluster.

■ Resume context: Configured HPAs in the e-commerce application on EKS.

Q: What is Kubernetes Ingress and how does it work?

- Ingress is a Kubernetes resource that manages external HTTP/HTTPS access to services within the cluster.
- It defines routing rules (host-based or path-based) to direct traffic to the appropriate Service.
- An Ingress Controller (e.g., AWS Load Balancer Controller, NGINX) is required to implement the Ingress rules — it reads the Ingress resource and configures the underlying load balancer.
- In the EKS project: the AWS Load Balancer Controller watched Ingress resources and automatically provisioned an AWS ALB.

- Example rule: ``host: api.example.com`` → routes to backend Service; ``host: app.example.com`` → routes to frontend Service.

■ Resume context: Used AWS Load Balancer Controller-based Ingress in the Three-Tier DevSecOps project.

Q: Troubleshooting: Pod is in CrashLoopBackOff state.

- Step 1 — Check pod logs: ``kubectl logs --previous`` (previous container instance logs).
 - Step 2 — Describe the pod: ``kubectl describe pod`` — check Events section for error details.
 - Step 3 — Common causes: application startup error, missing environment variable/secret, liveness probe too aggressive, OOMKilled (out of memory).
 - Step 4 — Check exit code in ``kubectl describe pod``: ExitCode 1 = application error; ExitCode 137 = OOMKilled.
 - Step 5 — If OOMKilled: increase memory limits in the pod spec. If config issue: verify ConfigMaps and Secrets are mounted correctly.
-

Q: Troubleshooting: Pod stuck in Pending state.

- Step 1 — Describe pod: ``kubectl describe pod`` and check the Events section.
 - Common reasons:
 - Insufficient resources: No node has enough CPU/memory to schedule the pod. Solution: scale node group or reduce pod requests.
 - Node selector / affinity mismatch: Pod requires a label not present on any node.
 - PVC not bound: Pod requires a PersistentVolumeClaim that is not yet bound.
 - Taint/toleration issue: Node is tainted and pod does not have the matching toleration.
-

Q: How did you manage Secrets in Kubernetes?

- Created Kubernetes Secrets to store sensitive data (DB passwords, API keys) in base64-encoded format.
- Referenced Secrets in pod specs as environment variables (``envFrom: secretRef``) or mounted as volumes.
- For AWS credentials, used IAM Roles for Service Accounts (IRSA) on EKS — attaching an IAM role to a Kubernetes Service Account, eliminating the need for hardcoded credentials.
- Best practice: never store secrets in Git. Use tools like AWS Secrets Manager or HashiCorp Vault for production-grade secret management.

■ Resume context: Managed Kubernetes Secrets and ConfigMaps in the e-commerce microservices deployment.

■ 5. AWS Interview Questions

Q: Explain EKS architecture.

- Amazon EKS (Elastic Kubernetes Service) is a managed Kubernetes service where AWS manages the Control Plane (API Server, etcd, Controller Manager, Scheduler).
- Worker Nodes run in your AWS account as EC2 instances (Managed Node Groups or self-managed) or as AWS Fargate pods.
- Managed Node Groups: AWS handles node provisioning, patching, and scaling; you define the instance type, AMI, and autoscaling config.
- EKS integrates natively with AWS services: ALB (Ingress), IAM (IRSA), ECR (image registry), CloudWatch (logging), and VPC (networking via VPC CNI).
- The EKS control plane is highly available across multiple Availability Zones.

■ *Resume context: Provisioned and managed EKS clusters using Terraform at Hisan Labs.*

Q: Difference between ALB and NLB?

- ALB (Application Load Balancer): Operates at Layer 7 (HTTP/HTTPS). Supports path-based and host-based routing, WebSockets, and sticky sessions. Ideal for web applications.
- NLB (Network Load Balancer): Operates at Layer 4 (TCP/UDP). Handles millions of requests per second with ultra-low latency. Ideal for real-time and high-performance workloads.
- ALB can inspect HTTP headers for routing decisions; NLB cannot.
- In Kubernetes: ALB is commonly used with Ingress (HTTP routing); NLB is used for Services of type LoadBalancer (TCP-level access).

■ *Resume context: Used AWS ALB with the AWS Load Balancer Controller for Kubernetes Ingress.*

Q: IAM User vs IAM Role — when to use which?

- IAM User: A permanent identity with long-term credentials (access key + secret). Used for human users or applications that need persistent credentials.
- IAM Role: A temporary identity assumed by AWS services, EC2 instances, Lambda functions, or cross-account entities. Credentials are short-lived and automatically rotated.
- Best practice: Use IAM Roles for all AWS services (EC2, EKS pods via IRSA, Lambda). Use IAM Users only for human console/CLI access, and enforce MFA.
- Least Privilege Principle: Grant only the permissions required for a specific task — nothing more.

■ *Resume context: Managed AWS IAM users, roles, and policies; implemented security best practices.*

Q: What is a VPC and explain its key components?

- VPC (Virtual Private Cloud): An isolated virtual network within AWS where you define IP address ranges, subnets, route tables, and security rules.
- Public Subnet: Has a route to the Internet Gateway. Resources here (e.g., ALB, NAT Gateway) are reachable from the internet.
- Private Subnet: No direct internet route. Resources (EKS nodes, RDS) run here for security.
- Internet Gateway: Enables internet access for public subnet resources.
- NAT Gateway: Allows private subnet resources to initiate outbound internet connections (e.g., downloading packages) without exposing them inbound.

- Security Groups: Stateful, instance-level firewall rules.
- NACLs: Stateless, subnet-level firewall rules evaluated in number order.

■ Resume context: Provisioned VPCs, subnets, and security groups using Terraform modules.

Q: What is CloudFront and how does caching work?

- AWS CloudFront is a Content Delivery Network (CDN) that caches content at Edge Locations globally, reducing latency for end users.
- When a user requests a resource, CloudFront checks its edge cache. On a cache hit, it serves from the edge. On a miss, it fetches from the origin (S3, ALB, EC2) and caches it.
- Cache behaviour is controlled by TTL (Time-to-Live) settings and Cache-Control headers from the origin.
- Cache invalidation: Force CloudFront to fetch fresh content by creating an invalidation (``/*` for all objects or specific paths).

■ Resume context: AWS CloudFront listed as part of the AWS skill set.

■ 6. Terraform Interview Questions

Q: What is Terraform remote state and why is it important?

- Terraform stores the state of managed infrastructure in a state file (terraform.tfstate).
- Remote state stores this file in a shared backend (e.g., AWS S3 with DynamoDB for locking) instead of locally.
- This enables team collaboration — all team members work against the same state.
- DynamoDB locking prevents two people from running `terraform apply` simultaneously, avoiding state corruption.
- State is sensitive (contains resource IDs, IPs, passwords in some cases) — store it encrypted in S3 with versioning enabled.

■ Resume context: Managed Terraform remote state configurations for collaborative infrastructure deployments.

Q: What are Terraform modules and why create them?

- A Terraform module is a collection of .tf files in a directory that encapsulates a reusable infrastructure component.
- Modules promote DRY (Don't Repeat Yourself) principles — define infrastructure once, reuse across environments (dev, staging, prod).
- A module accepts input variables and produces outputs, making it configurable and composable.
- Example: A `vpc` module that accepts CIDR block, availability zones, and environment name as inputs, and creates the complete VPC resource stack.

■ Resume context: Developed and modified Terraform modules for VPCs, Security Groups, S3, RDS, and EKS.

Q: Explain the Terraform workflow: init, plan, apply.

- `terraform init`: Initialises the working directory — downloads provider plugins and configures the backend.
 - `terraform plan`: Creates an execution plan — shows what changes will be made to infrastructure WITHOUT applying them. Always review this output.
 - `terraform apply`: Applies the planned changes. Prompts for confirmation unless `-auto-approve` is used.
 - `terraform destroy`: Destroys all managed resources — use with caution.
 - `terraform import`: Imports an existing real-world resource into Terraform state management.
-

Q: Scenario: Terraform apply fails halfway through. What do you do?

- Check the error output to identify which resource failed and why (e.g., insufficient IAM permissions, resource already exists).
- Run `terraform plan` again to see the current delta between state and real infrastructure.
- Fix the root cause (e.g., correct IAM policy, fix resource configuration).
- Re-run `terraform apply` — Terraform will only attempt to create/update resources that are not yet in the desired state.

- If a resource was partially created outside Terraform: use ``terraform import`` to bring it under state management.
 - Avoid manually modifying resources in the AWS console when using Terraform — this causes state drift.
-

Q: What is infrastructure drift and how do you handle it?

- Infrastructure drift occurs when the real-world infrastructure diverges from what is defined in Terraform state/code (e.g., someone manually modifies a Security Group in the console).
 - Detection: ``terraform plan`` will show the drift as a change even though you haven't modified any .tf files.
 - Resolution options: Either update the Terraform code to match the manual change (if it was intentional), or re-apply to revert the drift back to the desired state.
 - Prevention: Enforce Infrastructure as Code discipline — all changes must go through Terraform and version control.
-

■ 7. Jenkins & CI/CD Interview Questions

Q: Explain your Jenkins pipeline stage by stage.

- Stage 1 — Checkout: Jenkins pulls the latest code from GitHub triggered by a webhook on push/PR.
- Stage 2 — Build: Maven or npm compiles/builds the application and runs unit tests.
- Stage 3 — SonarQube Analysis: Code is submitted to SonarQube for static analysis. Quality Gate must pass to proceed.
- Stage 4 — OWASP Dependency Check: Scans application dependencies for known CVE vulnerabilities.
- Stage 5 — Docker Build: A Docker image is built from the multi-stage Dockerfile.
- Stage 6 — Trivy Scan: The Docker image is scanned for OS and library vulnerabilities.
- Stage 7 — Docker Push: The verified image is pushed to Amazon ECR.
- Stage 8 — Update Manifest: The Kubernetes deployment manifest is updated with the new image tag in the GitOps repository.
- Stage 9 — Argo CD Sync: Argo CD detects the manifest change and syncs the new version to the EKS cluster.

■ Resume context: Designed and maintained 3 Jenkins pipelines at Hisan Labs.

Q: Declarative vs Scripted Pipeline — what is the difference?

- Declarative Pipeline: Uses a structured, opinionated syntax with `pipeline {}` block. Easier to read, write, and validate. Supports Blue Ocean visualisation. Recommended for most use cases.
 - Scripted Pipeline: Uses Groovy DSL with full programmatic control. More flexible but harder to maintain and debug.
 - Declarative has built-in `options``, `parameters``, `triggers``, `post`` blocks. Scripted requires manual implementation of these.
 - Example: Declarative uses `stage('Build') { steps { sh 'mvn clean install' } }` whereas Scripted uses `node { stage('Build') { sh 'mvn clean install' } }`
-

Q: How does GitHub webhook trigger Jenkins?

- A GitHub webhook is configured on the repository to send an HTTP POST request to the Jenkins URL (`http://github-webhook/`) on specified events (push, pull_request).
- Jenkins has the GitHub Plugin installed and the job is configured with 'GitHub hook trigger for GITScm polling'.
- When code is pushed, GitHub sends the webhook payload to Jenkins; Jenkins validates the payload and triggers the matching pipeline.
- Troubleshooting webhook failures: Check GitHub repository Settings → Webhooks → recent deliveries for HTTP response codes. Ensure Jenkins is publicly reachable or use ngrok for local testing.

■ Resume context: Configured GitHub webhooks at Hisan Labs to trigger Jenkins pipelines.

Q: Jenkins build is failing — what do you check?

- Step 1 — Check the console output of the failed build for the exact error message.

- Step 2 — Identify the stage that failed (checkout, build, test, Docker build, etc.).
 - Step 3 — Common causes by stage:
 - Checkout: Git credentials expired, repository URL changed.
 - Build: Compile error, missing dependency, Maven/npm configuration issue.
 - SonarQube: Quality Gate failed (code coverage below threshold, critical bugs found).
 - Docker Build: Dockerfile syntax error, base image not found, insufficient disk space.
 - Docker Push: ECR credentials expired, IAM role missing ECR permissions.
 - Step 4 — Check Jenkins agent status: `Manage Jenkins → Manage Nodes`.
-

■ 8. GitOps & Argo CD Interview Questions

Q: What is GitOps and why use it?

- GitOps is an operational framework where Git is the single source of truth for both infrastructure and application configuration.
- All desired state is declared in Git repositories. An automated agent (like Argo CD) continuously ensures the live environment matches the Git state.
- Benefits: Full audit trail via Git history, easy rollbacks (revert a commit), improved consistency across environments, and reduced manual intervention.
- GitOps = declarative infrastructure + version control + automated reconciliation.

■ Resume context: Implemented GitOps-based application delivery using Argo CD in both projects.

Q: Explain Argo CD architecture and how it works.

- Argo CD is a declarative GitOps continuous delivery tool for Kubernetes.
- Components: API Server (CLI/UI access), Repository Server (clones Git repo, generates manifests), Application Controller (compares live vs desired state and reconciles).
- Workflow: Argo CD watches a Git repository (containing Kubernetes manifests or Helm charts). When a change is detected, it compares the Git state against the live cluster state.
- If drift is detected: in Auto-Sync mode, Argo CD automatically applies the changes. In Manual mode, it shows the diff and waits for operator approval.
- Rollback: Simply revert the Git commit — Argo CD will reconcile the cluster back to the previous state.

■ Resume context: Used Argo CD for automated synchronisation and deployment in both the Three-Tier and Go GitOps projects.

Q: How does your GitHub Actions + Argo CD pipeline work?

- Developer pushes code to GitHub → GitHub Actions workflow is triggered.
- GitHub Actions: runs tests, builds Docker image, tags it with commit SHA, pushes to ECR.
- GitHub Actions then updates the image tag in the Kubernetes manifest YAML in the GitOps repository (separate repo or branch).
- Argo CD continuously polls the GitOps repository (or uses a webhook). It detects the manifest change.
- Argo CD syncs the new manifest to the EKS cluster — Kubernetes performs a rolling update of the deployment.

■ Resume context: Go Cloud-Native GitOps Delivery Pipeline project.

■ 9. Monitoring & Logging Interview Questions

Q: Explain Prometheus architecture.

- Prometheus is a pull-based monitoring system that scrapes metrics from instrumented targets at regular intervals.

- Components: Prometheus Server (scrapes and stores time-series data), Alertmanager (handles alerts), Pushgateway (for short-lived jobs), Exporters (expose metrics from third-party systems).
- Exporters: node_exporter (hardware/OS metrics), kube-state-metrics (Kubernetes object metrics).
- Metrics are stored in a local time-series database and queried using PromQL.
- Grafana connects to Prometheus as a data source and visualises the metrics as dashboards.

■ Resume context: Configured Prometheus and Grafana using Helm charts in the Three-Tier DevSecOps project.

Q: CPU spikes suddenly in production — how do you investigate?

- Step 1 — Check Grafana dashboard: Identify the specific pod/node with the spike using CPU utilisation panels.
- Step 2 — Kubernetes: `kubectl top pods -n`` and `kubectl top nodes`` to pinpoint the resource.
- Step 3 — Check pod logs: `kubectl logs --tail=100`` for any error spike or runaway loop.
- Step 4 — Check CloudWatch Metrics for EC2 node-level CPU, or Datadog APM for application-level traces.
- Step 5 — If a specific process is culprit: exec into the pod (`kubectl exec -it -- bash``) and run `top``.
- Step 6 — Check for recent deployments that may have introduced the issue. Consider rolling back.
- Step 7 — If traffic-related: check if HPA has scaled out pods to distribute load.

■ Resume context: Monitored application and infrastructure performance using Prometheus, Grafana, Datadog, and AWS CloudWatch.

■ 10. DevSecOps & Security Questions

Q: What is DevSecOps and Shift Left Security?

- DevSecOps integrates security practices into every phase of the DevOps lifecycle rather than treating it as a final gate.
- Shift Left Security means moving security checks earlier in the pipeline (toward the 'left' side of the SDLC diagram) — catching issues during development rather than in production.
- In practice: static code analysis (SonarQube) runs on every commit; dependency scanning (OWASP) runs during build; container scanning (Trivy) runs before deployment.
- This reduces the cost and impact of fixing vulnerabilities — issues caught early are exponentially cheaper to fix.

■ Resume context: Integrated SonarQube, OWASP, and Trivy into Jenkins CI/CD pipeline.

Q: What does SonarQube check and how is it integrated?

- SonarQube performs static code analysis — it checks for bugs, code smells, security vulnerabilities, and code coverage.
- A Quality Gate defines pass/fail thresholds (e.g., code coverage > 80%, zero critical vulnerabilities).
- Integration: Jenkins uses the SonarQube Scanner plugin. The pipeline stage runs the scanner, submits the report to SonarQube server, and then polls the Quality Gate result.
- If the Quality Gate fails, the Jenkins pipeline is aborted — the image is not built or deployed.

■ Resume context: Integrated SonarQube into CI/CD pipelines for automated code quality analysis.

Q: What vulnerabilities does Trivy detect?

- Trivy is an open-source container image scanner that detects OS package vulnerabilities (CVEs), application dependency vulnerabilities, misconfigurations, and secrets in images.
- It scans the image layers against vulnerability databases (NVD, Red Hat, Ubuntu CVE trackers).
- Severity levels: CRITICAL, HIGH, MEDIUM, LOW, UNKNOWN.
- In the pipeline: after building the Docker image, Trivy scans it and can be configured to fail the build if CRITICAL or HIGH vulnerabilities are found.
- Using Distroless images significantly reduces Trivy findings by eliminating the OS user-space.

■ Resume context: Trivy container image scanning implemented prior to deployment in the Three-Tier project.

■ 11. Behavioural Interview Questions

Q: Tell me about a production issue you encountered and how you resolved it.

- Situation: During my internship, one of our backend services started returning 502 errors after a deployment.
- Task: I was responsible for investigating the root cause and restoring service.
- Action: I checked the ALB target group health checks — the new pods were failing health checks because the readiness probe was pointing to the wrong endpoint path after a recent code change. I identified this by reviewing the Kubernetes pod events (`kubectl describe pod`) and the ALB target group metrics in CloudWatch.

- Resolution: Updated the readiness probe path in the Kubernetes Deployment manifest, pushed the fix, and Argo CD automatically synced the update. Service was restored within 15 minutes.
 - Result: This incident led us to add health check endpoint validation as a mandatory CI pipeline step.
-

Q: Describe a challenge you faced during your internship.

- Challenge: Setting up Terraform remote state for the first time — I accidentally locked the state file while a failed apply was in progress.
 - The DynamoDB state lock table was still holding the lock, preventing any further Terraform operations.
 - Resolution: I learned about `terraform force-unlock` and used it to release the stale lock after confirming no other operation was running.
 - Learning: I documented the procedure and added a Slack notification step in the pipeline to alert the team when a Terraform lock was acquired, preventing future occurrences.
-

Q: Why DevOps? Why should we hire you?

- DevOps aligns perfectly with my interest in both software development and infrastructure — it sits at the intersection of development velocity and operational reliability.
 - I am drawn to the culture of automation, continuous improvement, and collaboration between teams.
 - Why hire me: I bring 9 months of hands-on experience with the exact tools used in modern cloud-native environments — AWS, Kubernetes, Terraform, Jenkins, Argo CD.
 - I have built and operated real CI/CD pipelines, not just academic projects. I understand the challenges of collaborative infrastructure management.
 - I am a fast learner, proactive in documentation, and committed to applying security best practices (DevSecOps) from day one.
-

■ Quick Reference: Most Asked Commands

Category	Command	Purpose
Kubernetes	kubectl get pods -A	List all pods in all namespaces
Kubernetes	kubectl describe pod <name>	Pod events and configuration
Kubernetes	kubectl logs <pod> --previous	Logs from crashed container
Kubernetes	kubectl exec -it <pod> -- bash	Shell into running container
Kubernetes	kubectl top pods / nodes	CPU & memory usage
Docker	docker logs <container>	Container stdout/stderr logs
Docker	docker inspect <container>	Container metadata & config
Docker	docker system prune -af	Remove unused images/containers
Linux	df -h	Disk usage by filesystem
Linux	du -sh /* sort -rh	Find largest directories
Linux	ps aux --sort=-%cpu	Processes sorted by CPU
Linux	journalctl -xe	System journal with errors
Terraform	terraform plan	Preview infrastructure changes
Terraform	terraform state list	List managed resources
Terraform	terraform force-unlock <ID>	Release stale state lock
AWS CLI	aws eks update-kubeconfig	Configure kubectl for EKS
AWS CLI	aws ecr get-login-password ...	Authenticate Docker to ECR

Prepared based on Sanket Chopade's resume and internship at Hisan Labs (Sep 2025 – May 2026). All answers are tailored to real project experience.